

A BUILD-BY-DOING TUTORIAL

FastAPI Zero to Production

From “what is an API?” to a deployed, tested, authenticated backend — one phase at a time.

4

PHASE FOUR

Database Integration with SQLAlchemy

Replace the in-memory store with a real database: engine and sessions, ORM models with relationships, Alembic migrations, clean CRUD, dependency injection, and a one-line switch from SQLite to PostgreSQL.

Contents

4.1 What Is a Database and Why Do We Need One?	4
RAM vs disk — the core reason	4
Relational vs NoSQL	5
SQLite for development, PostgreSQL for production	5
What is an ORM?	5
4.2 SQLAlchemy Setup	5
4.3 Creating the Database Engine and Session	6
4.4 SQLAlchemy Models	8
Column options you'll use constantly	8
Relationships: one-to-many	9
4.5 Database Migrations with Alembic	9
Why migrations?	9
Setting up Alembic	10
Creating and applying migrations	10
4.6 CRUD Operations	11
4.7 Dependency Injection with Depends()	13
What is dependency injection?	13
Why this is so powerful	14
Tying it together in main.py	14
4.8 Switching to PostgreSQL	14
Hands-On Exercise 4: Book Library with SQLite	15
Checkpoint: At This Point You Can Now...	18
Phase 4 Summary	19

Your Journey

✓	Foundations: What Is an API?	DONE
✓	Setup & Your First API	DONE
✓	Path Params, Query Params & Body	DONE
✓	Pydantic: Data Validation	DONE
4	Database Integration (SQLAlchemy)	YOU ARE HERE
5	Authentication & Authorization	
6	Advanced FastAPI Features	
7	Testing Your FastAPI App	
8	Deployment with Docker	
★	Capstone — TaskFlow API	

Until now your data has lived in a Python list or dict — which means it vanishes the instant the server restarts. That's fine for learning, useless for anything real. In this phase you give your API permanent memory. This is a meatier phase with more moving parts, so we'll go step by step and keep a clear project structure throughout. Type along, run things, and let the pieces click into place one at a time.

4.1 What Is a Database and Why Do We Need One?

RAM vs disk — the core reason

Your in-memory list lives in **RAM** — fast, but erased when the program stops. A database stores data on **disk**, so it persists across restarts, crashes, and deployments. That permanence is the entire point.

A database also gives you things a Python list never could: efficient searching over millions of rows, guarantees that data stays consistent even with many simultaneous users, relationships between different kinds of data, and the ability for *other* programs to read the same data safely.

Relational vs NoSQL

There are two broad families. **Relational** (SQL) databases store data in tables of rows and columns, with strict structure and powerful relationships — PostgreSQL, MySQL, SQLite. **NoSQL** databases (MongoDB, Redis) trade some structure for flexibility or speed. For the vast majority of APIs — and this whole tutorial — a **relational** database is the right, safe default. We'll use SQL.

SQLite for development, PostgreSQL for production

- **SQLite** is a tiny database that lives in a single file on disk, with zero setup. Perfect for development and learning.
- **PostgreSQL** is a powerful, production-grade server database that handles heavy concurrent load. Perfect for the real world.

The beautiful part, which you'll see in 4.8: with SQLAlchemy in between, *your application code is nearly identical for both*. You develop on SQLite and deploy on PostgreSQL by changing one line.

What is an ORM?

Talking to a database means writing SQL (`SELECT * FROM books WHERE id = 42`). An **ORM** (Object-Relational Mapper) lets you work with **Python classes and objects instead of raw SQL**. You define a `Book` class; SQLAlchemy maps it to a `books` table. You create a `Book()` object and add it; SQLAlchemy writes the `INSERT`. The analogy: **a Python class is a table, an instance is a row, an attribute is a column**. SQLAlchemy is the most popular Python ORM, and the one FastAPI projects use most.

CONCEPT CHECK

Concept Check: Why develop on SQLite but deploy on PostgreSQL? *Answer: SQLite needs zero setup so it's frictionless for local development, while PostgreSQL handles real concurrent production load. SQLAlchemy abstracts the difference, so you get the convenience of both without rewriting your code.*

4.2 SQLAlchemy Setup

Install SQLAlchemy, Alembic (for migrations), and the PostgreSQL driver (for later):

BASH / TERMINAL

```
# sqlalchemy = the ORM; alembic = migrations; psycopg2-binary = PostgreSQL driver
pip install sqlalchemy alembic psycopg2-binary
```

As the project grows, a flat `main.py` won't scale. Adopt this structure now — it's the standard layout you'll use all the way to the capstone:

TEXT

```
book-library-api/
├── app/
│   ├── __init__.py
│   ├── main.py           ← FastAPI app + route includes
│   ├── database.py       ← engine, SessionLocal, Base, get_db
│   ├── models.py         ← SQLAlchemy ORM models (tables)
│   ├── schemas.py        ← Pydantic schemas (validation)
│   ├── crud.py           ← database read/write functions
│   └── routers/
│       ├── __init__.py
│       └── books.py       ← book endpoints
└── alembic/              ← migration scripts (created in 4.5)
```

TIP

Tip — schemas vs models, an important distinction: From now on we use two parallel words carefully. **Models** (in `models.py`) are *SQLAlchemy* classes — they describe database **tables**. **Schemas** (in `schemas.py`) are *Pydantic* classes — they describe the **JSON** clients send and receive. They look similar but do different jobs: models talk to the database, schemas talk to the outside world. Keeping them separate is a hallmark of clean FastAPI code.

4.3 Creating the Database Engine and Session

Everything database-related starts in `database.py`. Here's the whole file, explained:

PYTHON

```
# app/database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

# The DATABASE_URL tells SQLAlchemy where and how to connect.
# Format: dialect+driver://user:password@host:port/dbname
# For SQLite it's just a file path. (We move this to .env in Phase 6.)
DATABASE_URL = "sqlite:///./books.db"

# The engine is the core connection manager to the database.
# check_same_thread is a SQLite-only quirk needed for FastAPI's threading.
engine = create_engine(
    DATABASE_URL, connect_args={"check_same_thread": False}
)

# A session is your "conversation" with the DB for one unit of work.
# SessionLocal is a factory that produces new sessions on demand.
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# Base is the parent class all ORM models inherit from. SQLAlchemy uses
# it to keep a registry of every table.
Base = declarative_base()

# This dependency provides a database session to a route, then guarantees
# it's closed afterward – even if the route raises an error.
def get_db():
    db = SessionLocal()
    try:
        yield db          # hand the session to the route function
    finally:
        db.close()       # always run, cleaning up the connection
```

The three key objects:

- **engine** — manages the actual connection pool to the database. Created once.
- **SessionLocal** — a factory; calling `SessionLocal()` gives you a fresh **session**, your workspace for a set of reads/writes.
- **Base** — the declarative base every model subclasses, so SQLAlchemy knows about all your tables.

And `get_db()` is a **dependency** (more on these in 4.7). The `yield` is the clever bit: code before `yield` runs to set up the session, the session is used by the route, and code after `yield` (in `finally`) runs to tear it down — closing the connection no matter what happens.

COMMON MISTAKES

Common Mistakes: The `connect_args={"check_same_thread": False}` is **only** for SQLite. When you switch to PostgreSQL in 4.8, remove it — PostgreSQL doesn't need or want it.

4.4 SQLAlchemy Models

A model is a Python class mapping to a table. Define them in `models.py`:

PYTHON

```
# app/models.py
from sqlalchemy import Column, Integer, String, Boolean, ForeignKey
from sqlalchemy.orm import relationship
from app.database import Base

class User(Base):
    __tablename__ = "users"          # the table's name in the database

    id = Column(Integer, primary_key=True, index=True)
    username = Column(String, unique=True, nullable=False)
    is_active = Column(Boolean, default=True)

    # One user has many books. `books` is a convenience attribute, not a column.
    # back_populates links it to the matching attribute on Book.
    books = relationship("Book", back_populates="owner")

class Book(Base):
    __tablename__ = "books"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String, nullable=False, index=True)
    author = Column(String, nullable=False)
    year = Column(Integer)

    # The foreign key column: links each book to a row in users.
    owner_id = Column(Integer, ForeignKey("users.id"))

    # The reverse side of the relationship defined on User.
    owner = relationship("User", back_populates="books")
```

Column options you'll use constantly

Option	Meaning
<code>primary_key=True</code>	The unique identifier column (usually <code>id</code>)
<code>index=True</code>	Build an index for fast lookups/filtering on this column
<code>nullable=False</code>	The column is required (cannot be NULL)
<code>unique=True</code>	No two rows may share this value (e.g. <code>username</code>)
<code>default=...</code>	Value used when none is provided

Relationships: one-to-many

A `User` has many `Book`s; each `Book` belongs to one `User`. This is a **one-to-many** relationship, expressed by:

1. A `ForeignKey("users.id")` column on the "many" side (`Book.owner_id`).
2. A `relationship(...)` on **both** sides, linked with `back_populates`.

With this in place you can write `a_user.books` to get all of a user's books, and `a_book.owner` to get a book's user — SQLAlchemy generates the SQL joins for you.

TIP

Tip — many-to-many (preview): When both sides can have many of the other (a book has many tags, a tag belongs to many books), you add an **association table** and use `secondary=...` in the relationship. You'll build exactly this for `Task↔Tag` in the capstone. One-to-many (a foreign key on the child) covers most cases until then.

CONCEPT CHECK

Concept Check: On which table does the `ForeignKey` go — `User` or `Book`? Answer: On `Book` (the "many" side), as `owner_id`. The foreign key always lives on the child that points back to its single parent.

4.5 Database Migrations with Alembic

Why migrations?

Your model classes define the *desired* schema, but the database needs matching tables actually created — and when you later add a column, you need to change the live database *without losing existing data*. **Migrations** are versioned scripts that evolve your schema safely and reproducibly. Alembic is SQLAlchemy's migration tool.

WARNING

Warning: For quick experiments you might see `Base.metadata.create_all(bind=engine)` used to create tables directly. That's fine for throwaway scripts, but it can't evolve a schema (add/rename columns on an existing database) without dropping data. For anything real, use Alembic from the start.

Setting up Alembic

BASH / TERMINAL

```
# Initialize Alembic – creates an alembic/ folder and alembic.ini
alembic init alembic
```

Then make two edits so Alembic knows about your database and models:

1. In `alembic.ini`, set the URL (or leave it and set it in `env.py`):

TEXT

```
sqlalchemy.url = sqlite:///./books.db
```

1. In `alembic/env.py`, point `target_metadata` at your `Base` so autogenerate can see your models:

PYTHON

```
# alembic/env.py (near the top, after imports)
from app.database import Base
from app import models          # ensure models are imported so they register
target_metadata = Base.metadata
```

Creating and applying migrations

BASH / TERMINAL

```
# Autogenerate a migration by comparing your models to the current DB.
alembic revision --autogenerate -m "create users and books tables"

# Apply all pending migrations, bringing the DB up to the latest schema.
alembic upgrade head
```

`upgrade head` runs the migration and creates your tables. Later, when you change a model, you run `revision --autogenerate` again to capture the diff, then `upgrade head` to apply it.

To undo the most recent migration:

BASH / TERMINAL

```
# Roll back exactly one migration.
alembic downgrade -1
```

TIP

Tip: Always *read* the autogenerated migration before applying it. Autogenerate is good but not perfect — it sometimes misses renames (seeing them as a drop + add) or server-side defaults. A quick scan of the generated `upgrade()` function saves painful surprises.

4.6 CRUD Operations

Keep all database read/write logic in `crud.py`, separate from your routes. This makes the logic reusable and testable, and keeps endpoints thin. CRUD = **C**reate, **R**ead, **U**ppdate, **D**elete.

```

# app/crud.py
from sqlalchemy.orm import Session
from app import models, schemas

def create_book(db: Session, book: schemas.BookCreate, owner_id: int):
    # Build a SQLAlchemy model instance from the validated schema data.
    db_book = models.Book(**book.model_dump(), owner_id=owner_id)
    db.add(db_book)          # stage the insert
    db.commit()              # write it to the database
    db.refresh(db_book)      # reload so we get the DB-assigned id
    return db_book

def get_book(db: Session, book_id: int):
    # .first() returns the row or None.
    return db.query(models.Book).filter(models.Book.id == book_id).first()

def get_books(db: Session, skip: int = 0, limit: int = 100):
    # offset/limit give us pagination over the table.
    return db.query(models.Book).offset(skip).limit(limit).all()

def update_book(db: Session, book_id: int, data: schemas.BookUpdate):
    db_book = get_book(db, book_id)
    if db_book is None:
        return None
    # Only update fields the client actually sent (exclude_unset).
    for field, value in data.model_dump(exclude_unset=True).items():
        setattr(db_book, field, value)
    db.commit()
    db.refresh(db_book)
    return db_book

def delete_book(db: Session, book_id: int):
    db_book = get_book(db, book_id)
    if db_book is None:
        return False
    db.delete(db_book)
    db.commit()
    return True

```

The pattern is always the same trio for writes: **add / delete** → **commit** → **refresh**. **add** stages the change, **commit** makes it permanent, **refresh** reloads the object so attributes like the new **id** are populated.

COMMON MISTAKES

Common Mistakes: Forgetting `db.commit()`. Without it, your `add` or `delete` is staged but never written, and the change silently disappears when the session closes. If a write "doesn't save," check for a missing commit first.

4.7 Dependency Injection with `Depends()`

What is dependency injection?

Dependency injection sounds fancy but means something simple: *a function declares what it needs, and the framework provides it*. Instead of each route building its own database session, a route just says "I need a `db` session," and FastAPI calls `get_db()` and hands one over. The analogy: you order "a coffee" (declare a need); you don't grind beans and operate the machine yourself (build it). FastAPI is the barista.

You wire it up with `Depends()`:

PYTHON

```
# app/routers/books.py
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from app import crud, schemas
from app.database import get_db

router = APIRouter(prefix="/books", tags=["books"])

@router.post("", response_model=schemas.BookResponse,
             status_code=status.HTTP_201_CREATED)
def create_book(book: schemas.BookCreate, db: Session = Depends(get_db)):
    # FastAPI calls get_db(), passes the session in as `db`, closes it after.
    return crud.create_book(db, book, owner_id=1)

@router.get("", response_model=list[schemas.BookResponse])
def list_books(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    return crud.get_books(db, skip=skip, limit=limit)

@router.get("/{book_id}", response_model=schemas.BookResponse)
def read_book(book_id: int, db: Session = Depends(get_db)):
    db_book = crud.get_book(db, book_id)
    if db_book is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
                             detail="Book not found")
    return db_book
```

The single line `db: Session = Depends(get_db)` is the whole mechanism: it gives this route a ready-to-use, auto-closed session. Every route that needs the database declares the same dependency.

Why this is so powerful

Dependencies are **reusable** and **composable**. The same `get_db` works in every route. In Phase 5 you'll write a `get_current_user` dependency for authentication, and routes will simply declare `user = Depends(get_current_user)` to require a logged-in user. Dependencies can even depend on *other* dependencies (nested), so `get_current_user` itself uses `get_db`. This is the backbone of clean FastAPI architecture.

TIP

Tip: Because dependencies are swappable, you can **override** them in tests — replacing `get_db` with one that points at a throwaway test database. That single capability makes FastAPI apps a joy to test, as you'll see in Phase 7.

Tying it together in `main.py`

PYTHON

```
# app/main.py
from fastapi import FastAPI
from app.routers import books

app = FastAPI(title="Book Library API")

# Mount the books router; all its routes get the /books prefix.
app.include_router(books.router)
```

4.8 Switching to PostgreSQL

Here's the payoff for using an ORM. To move from SQLite to PostgreSQL:

1. Run a PostgreSQL server (locally, or via Docker — covered in Phase 8).
2. Change **one line** — the `DATABASE_URL`:

PYTHON

```
# Before (SQLite):  
DATABASE_URL = "sqlite:///./books.db"  
  
# After (PostgreSQL):  
DATABASE_URL = "postgresql://user:password@localhost:5432/booklib"
```

1. Remove the SQLite-only `connect_args={"check_same_thread": False}`.
2. Run your migrations against the new database: `alembic upgrade head`.

That's it. Your models, schemas, CRUD functions, and routes don't change at all. **That** is the power of the abstraction: you wrote your app against SQLAlchemy, not against a specific database, so swapping the database is a configuration change, not a rewrite.

CONCEPT CHECK

Concept Check: After switching the URL to PostgreSQL, how much of your `crud.py` needs to change? *Answer: None of it. CRUD functions speak SQLAlchemy, which speaks to whatever database the engine is configured for. That's the entire benefit of the ORM abstraction.*

HANDS-ON

Hands-On Exercise 4: Book Library with SQLite

Wire the whole thing together: `User` and `Book` models with a one-to-many relationship, full CRUD, pagination, and an endpoint listing a user's books. Below is a single-file version (`app.py`) you can run immediately — it uses `create_all` for simplicity so you can see it work end-to-end; in a real project you'd split files and use Alembic as shown above.

```

# app.py - runnable single-file demo
from fastapi import FastAPI, Depends, HTTPException, status
from sqlalchemy import create_engine, Column, Integer, String, ForeignKey
from sqlalchemy.orm import sessionmaker, declarative_base, relationship, Session
from pydantic import BaseModel, Field

# ---- database setup ----
engine = create_engine("sqlite:///./books.db",
                      connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(bind=engine, autoflush=False)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# ---- ORM models ----
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    username = Column(String, unique=True, nullable=False)
    books = relationship("Book", back_populates="owner")

class Book(Base):
    __tablename__ = "books"
    id = Column(Integer, primary_key=True, index=True)
    title = Column(String, nullable=False, index=True)
    author = Column(String, nullable=False)
    year = Column(Integer)
    owner_id = Column(Integer, ForeignKey("users.id"))
    owner = relationship("User", back_populates="books")

Base.metadata.create_all(bind=engine)  # create tables (demo only)

# ---- Pydantic schemas ----
class UserCreate(BaseModel):
    username: str = Field(min_length=3, max_length=50)

class UserResponse(BaseModel):
    id: int
    username: str
    model_config = {"from_attributes": True}

class BookCreate(BaseModel):
    title: str = Field(min_length=1, max_length=200)
    author: str
    year: int = Field(ge=1000, le=2025)

class BookResponse(BookCreate):
    id: int
    owner_id: int
    model_config = {"from_attributes": True}

# ---- app + routes ----
app = FastAPI(title="Book Library API (SQLite)")

```

```

@app.post("/users", response_model=UserResponse, status_code=201)
def create_user(payload: UserCreate, db: Session = Depends(get_db)):
    user = User(username=payload.username)
    db.add(user); db.commit(); db.refresh(user)
    return user

@app.post("/users/{user_id}/books", response_model=BookResponse,
          status_code=201)
def create_book(user_id: int, payload: BookCreate,
                db: Session = Depends(get_db)):
    if db.get(User, user_id) is None:
        raise HTTPException(404, "User not found")
    book = Book(**payload.model_dump(), owner_id=user_id)
    db.add(book); db.commit(); db.refresh(book)
    return book

@app.get("/books", response_model=list[BookResponse])
def list_books(skip: int = 0, limit: int = 10, db: Session = Depends(get_db)):
    return db.query(Book).offset(skip).limit(limit).all()

@app.get("/users/{user_id}/books", response_model=list[BookResponse])
def user_books(user_id: int, db: Session = Depends(get_db)):
    user = db.get(User, user_id)
    if user is None:
        raise HTTPException(404, "User not found")
    return user.books  # the relationship gives us all their books

@app.get("/books/{book_id}", response_model=BookResponse)
def get_book(book_id: int, db: Session = Depends(get_db)):
    book = db.get(Book, book_id)
    if book is None:
        raise HTTPException(404, "Book not found")
    return book

```

Run and test:

BASH / TERMINAL

```
uvicorn app:app --reload
```

TEXT

POST /users	{"username": "ada"}	→ 201, user id 1
POST /users/1/books	{"title": "Dune", "author": "Herbert", "year": 1965}	→ 201
POST /users/1/books	{"title": "1984", "author": "Orwell", "year": 1949}	→ 201
GET /users/1/books		→ both books
GET /books?skip=0&limit=1		→ first book only (pagination)
GET /books/99		→ 404 Book not found

Inspect the file. After running, you'll find a `books.db` file in your folder. Open it with the free **DB Browser for SQLite** to see your `users` and `books` tables with real rows — proof the data now lives on disk and survives restarts.

What just happened? Your API now has permanent memory. The `User / Book` one-to-many relationship lets `GET /users/1/books` return a user's books via `user.books` — SQLAlchemy writes the join for you. `skip / limit` give you pagination. Every route gets a database session through `Depends(get_db)`, which is opened and reliably closed for each request. Stop the server, restart it, and your data is still there. You've crossed the line from toy to real application.

COMMON MISTAKES

Common Mistakes: Returning a SQLAlchemy model directly without a Pydantic `response_model` that has `from_attributes=True`. Without it, FastAPI can't read the ORM object's attributes and you'll get a serialization error. Every response model for ORM data needs `model_config = {"from_attributes": True}`.

TIP

Tip: In this demo we hardcoded table creation with `create_all`. In the capstone (and any real project) you'll use the `app/` package structure and Alembic migrations from 4.5, so schema changes are versioned and safe. The *concepts* are identical; only the file organisation and the table-creation mechanism differ.

Checkpoint: At This Point You Can Now...

- ☐ Explain **why a database** is needed (persistence) and the difference between **models** (SQLAlchemy/tables) and **schemas** (Pydantic/JSON).
- ☐ Set up `database.py` with an **engine**, `SessionLocal`, `Base`, and a `get_db` dependency.
- ☐ Define **SQLAlchemy models** with column options and a **one-to-many relationship** (`ForeignKey` + `relationship` + `back_populates`).
- ☐ Use **Alembic** to autogenerate and apply migrations (`revision --autogenerate`, `upgrade head`, `downgrade -1`).
- ☐ Write **CRUD** functions using the `add/delete → commit → refresh` pattern with pagination.
- ☐ Inject the database into routes with `Depends(get_db)` and explain dependency injection.
- ☐ Switch from **SQLite to PostgreSQL** by changing only the `DATABASE_URL`.

Phase 4 Summary

PHASE 4 • KEY TAKEAWAYS

In this phase your API gained permanent memory with a real database.

- Data in a list lives in **RAM** and vanishes on restart; a **database** stores it on **disk**. Relational/SQL is the safe default; **SQLite** for dev, **PostgreSQL** for production.
- An **ORM** (SQLAlchemy) maps **Python classes** → **tables**, **instances** → **rows**, **attributes** → **columns**, so you write Python instead of raw SQL.
- `database.py` holds the **engine**, the `SessionLocal` factory, the `Base` for models, and the `get_db` dependency that opens and reliably closes a session per request.
- **Models** describe tables; **one-to-many** relationships use a `ForeignKey` on the child plus `relationship(..., back_populates=...)` on both sides.
- **Alembic** versions your schema: `revision --autogenerate` captures changes, `upgrade head` applies them, `downgrade -1` reverts.
- **CRUD** functions follow `add/delete → commit → refresh`; routes stay thin and get their session via `Depends(get_db)`.
- Switching databases is a **one-line** `DATABASE_URL` **change** — the payoff of coding against the ORM, not a specific database.
- **You rebuilt the Book Library on SQLite** with `User`/`Book`, a one-to-many relationship, full CRUD, pagination, and a per-user books endpoint — and the data survives restarts.

Your data is now persistent and relational. Next phase, you'll lock it down with real JWT authentication and authorization.

End of Phase 4. Reply "**continue**" for Phase 5 — Authentication & Authorization — where you secure the API with password hashing and JWT tokens.